

Sviluppo di interfacce web con Flask

Questa dispensa introduce lo sviluppo della parte di interfaccia web usando Flask. Il taglio e' progressivo: l'obiettivo non e' imparare subito tutto lo stack web, ma costruire una mappa mentale solida per leggere e modificare una piccola applicazione.

Il caso di riferimento e' l'applicazione palestra gia' presente nel repository. La stessa base dati e le stesse operazioni CRUD vengono rese disponibili tramite una interfaccia web fatta di pagine, link e form.

1. Che cosa significa interfaccia web

Una interfaccia web e' il punto di contatto tra utente e applicazione.

In una applicazione testuale l'utente interagisce con menu e prompt. In una applicazione web l'utente interagisce con:

- pagine HTML
- link
- pulsanti
- form
- messaggi
- tabelle visualizzate nel browser

Questi elementi grafici non sono oggetti "magici": sono rappresentati da codice. Il browser riceve HTML, CSS e talvolta JavaScript, li interpreta e costruisce una pagina interattiva.

Nel laboratorio palestra:

- la home mostra tre pannelli: amministratore, istruttore, iscritto;
- ogni pannello e' un link;
- le liste mostrano righe lette dal database;
- i form raccolgono dati;
- i pulsanti inviano richieste al server Flask.

2. Client, server e richiesta HTTP

Una applicazione web semplice coinvolge almeno due parti:

- client: il browser dell'utente;
- server: il programma Flask.

Il browser non esegue direttamente il codice Python dell'applicazione. Il browser fa richieste HTTP. Flask riceve una richiesta, esegue codice Python, eventualmente interroga il database, costruisce una risposta HTML e la restituisce al browser.

Schema operativo:

```
utente
  -> browser
    -> richiesta HTTP
      -> route Flask
        -> query / logica applicativa
          -> template HTML
            -> risposta HTTP
              -> browser
                -> DOM visualizzato
```

Questa sequenza e' essenziale: il codice Python vive sul server, mentre la pagina visualizzata vive nel browser.

3. HTML, CSS e JavaScript

In una interfaccia web si incontrano spesso tre linguaggi con responsabilita' diverse.

HTML

HTML descrive la struttura della pagina.

Esempio:

```
<a href="/admin">Amministratore</a>
```

Questo frammento descrive un link. Il browser lo trasforma in un oggetto cliccabile.

CSS

CSS descrive la presentazione.

Esempio:

```
.panel {
  border-radius: 16px;
  padding: 1.5rem;
}
```

Il CSS non decide quali dati mostrare. Decide come gli elementi vengono presentati.

JavaScript

JavaScript consente di reagire a eventi nel browser senza necessariamente ricaricare la pagina.

In questa lezione lo introduciamo come concetto, ma l'app palestra resta volutamente semplice: molte interazioni avvengono tramite link e form HTML standard.

4. Il DOM

DOM significa Document Object Model.

Quando il browser riceve HTML, non lo tratta solo come testo. Lo trasforma in una struttura ad albero di oggetti. Ogni tag HTML diventa un nodo del DOM.

Esempio HTML:

```
<section class="hero">
  <h1>Applicazione web per la gestione palestra</h1>
  <a href="/admin">Amministratore</a>
</section>
```

Rappresentazione concettuale:

```
document
  html
    body
      section.hero
        h1
        a
```

Il DOM e' importante perche' e' la rappresentazione interna con cui il browser ragiona sulla pagina. Quando si clicca un link, si compila un form o si modifica un campo, l'evento avviene su un nodo del DOM.

5. Mappare oggetti grafici e oggetti nel codice

Un errore comune e' pensare che la pagina sia separata dal codice.

In realta', ogni oggetto grafico ha una rappresentazione testuale e una responsabilita' applicativa.

Oggetto visibile	Codice HTML	Significato applicativo
Link	<code></code>	naviga verso una route
Pulsante di invio	<code><button type="submit"></code>	invia un form
Campo testo	<code><input name="nome"></code>	produce un valore nella richiesta
Tabella	<code><table></code>	mostra una collezione di righe
Messaggio	<code><p class="message"></code>	comunica esito o errore

Nel laboratorio palestra, il link "Amministratore" nella home non e' solo una scritta: e' un elemento `<a>` il cui `href` punta alla route Flask che mostra il menu amministratore.

6. Eventi

Un evento e' qualcosa che accade durante l'interazione.

Esempi:

- l'utente clicca un link;
- l'utente preme un pulsante;
- l'utente modifica il valore di un campo;
- un form viene inviato;
- la pagina viene caricata.

In una applicazione web semplice, molti eventi producono una nuova richiesta HTTP.

Esempio:

```
click su "Amministratore"  
-> richiesta GET /admin  
    -> Flask esegue admin_home()  
        -> Flask restituisce role.html
```

Nel caso di un form:

```
submit del form nuovo iscritto  
-> richiesta POST /entity/iscritti/new  
    -> Flask legge request.form  
        -> esegue INSERT  
            -> redirect alla lista iscritti
```

7. Flask: il ruolo delle route

In Flask, una route collega un URL a una funzione Python.

Esempio semplificato:

```
@app.get("/admin")  
def admin_home():  
    return render_template("role.html", role="amministratore")
```

La route dice:

- quando arriva una richiesta GET /admin;
- esegui la funzione admin_home;
- costruisci una risposta usando un template.

Una route e' quindi il punto di ingresso lato server per una interazione web.

8. Template Jinja

Flask usa spesso Jinja per generare HTML dinamico.

Un template e' un file HTML con parti variabili.

Esempio:

```
<h1>{{ role }}</h1>
```

Se Flask invoca:

```
render_template("role.html", role="amministratore")
```

il browser riceverà HTML con il valore concreto:

```
<h1>amministratore</h1>
```

La distinzione è importante:

- Jinja viene valutato sul server;
- il browser riceve solo HTML già prodotto;
- il DOM nasce dall'HTML finale, non dal template originale.

9. Ereditarietà dei template

Nel laboratorio palestra il file `layout.html` definisce la struttura comune:

- dichiarazione HTML;
- intestazione;
- link al foglio di stile;
- barra di navigazione;
- area principale della pagina;
- blocco `content`.

Le altre pagine estendono il layout:

```
{% extends "layout.html" %}
```

```
{% block content %}
```

```
...
```

```
{% endblock %}
```

Questa scelta evita di ripetere la stessa struttura in ogni pagina. Dal punto di vista architetturale, è una forma di riuso.

10. Form e dati in ingresso

Un form HTML raccoglie valori e li invia al server.

Esempio concettuale:

```
<form method="post">
  <input name="nome">
  <input name="cognome">
  <button type="submit">Salva</button>
</form>
```

Quando l'utente preme "Salva", il browser costruisce una richiesta HTTP. Flask legge i valori usando `request.form`.

Esempio:

```
nome = request.form["nome"].strip()
cognome = request.form["cognome"].strip()
```

Questo collega l'oggetto grafico al codice:

```
campo input name="nome"
  -> valore inviato nella richiesta
      -> request.form["nome"]
          -> parametro della query INSERT o UPDATE
```

11. GET e POST

Nelle applicazioni introduttive e' utile distinguere due metodi HTTP.

GET

GET legge una risorsa.

Esempi:

- aprire la home;
- visualizzare la lista degli iscritti;
- aprire il form di modifica;
- consultare il dettaglio di una scheda.

POST

POST invia dati che possono modificare lo stato del sistema.

Esempi:

- creare un iscritto;
- aggiornare un esercizio;
- eliminare una riga;
- registrare una esecuzione.

Regola pratica:

GET per leggere, POST per modificare.

12. Redirect e messaggi

Dopo una modifica, una route Flask spesso non restituisce direttamente la pagina finale. Esegue invece un redirect.

Esempio:

```
flash("Record aggiornato.", "success")
return redirect(url_for("entity_list", name=name))
```

Il redirect evita che il browser reinvii accidentalmente la stessa modifica se l'utente aggiorna la pagina. Il messaggio **flash** informa l'utente sull'esito dell'operazione.

Questa sequenza e' nota come pattern Post/Redirect/Get:

```
POST modifica dati
  -> redirect
    -> GET pagina aggiornata
```

13. Dal database alla pagina

Il passaggio da database a DOM avviene in piu' passaggi.

Esempio: mostrare le schede.

```
tabella SchedeAllenamento
  -> query SQL con JOIN
    -> lista di righe Python
      -> render_template(...)
        -> HTML
          -> DOM nel browser
            -> tabella visibile
```

Questa catena aiuta a capire dove cercare gli errori:

- se mancano dati nel database, la query non li restituisce;
- se la query e' corretta ma la pagina no, il problema puo' essere nel template;
- se il template e' corretto ma la pagina appare male, il problema puo' essere nel CSS.

14. Route generiche nel laboratorio palestra

L'app palestra usa una scelta interessante: alcune pagine CRUD sono gestite da route generiche.

Esempio:

```
@app.get("/entity/<name>")
def entity_list(name):
    ...
```

Il parametro **name** consente di usare una sola funzione per entita' diverse, come **iscritti**, **istruttori** ed **esercizi**.

Questa scelta e' didatticamente utile perche' mostra che:

- la pagina visibile puo' cambiare;
- la struttura dell'operazione resta simile;

- una configurazione Python puo' descrivere colonne, campi e tabella da usare.

15. Sicurezza e responsabilita'

In una lezione introduttiva non costruiamo ancora autenticazione completa, ma e' importante vedere alcune responsabilita'.

Un'interfaccia web dovrebbe:

- validare i dati ricevuti;
- gestire errori di vincolo;
- evitare operazioni distruttive involontarie;
- non fidarsi ciecamente del browser;
- distinguere i ruoli applicativi.

Nel laboratorio palestra i ruoli sono simulati tramite navigazione separata. In una applicazione reale servirebbero login, sessioni e autorizzazioni.

16. Lettura guidata dell'app palestra web

Per leggere l'app, partire da questi file:

- `labs/palestra_web/app.py`: route, query e operazioni;
- `labs/palestra_web/templates/layout.html`: struttura comune;
- `labs/palestra_web/templates/index.html`: home;
- `labs/palestra_web/templates/entity_list.html`: lista generica;
- `labs/palestra_web/templates/entity_form.html`: form generico;
- `labs/palestra_web/static/style.css`: stile.

Sequenza consigliata:

1. aprire la home;
2. trovare la route Flask corrispondente;
3. trovare il template usato;
4. individuare gli elementi HTML prodotti;
5. capire quale evento utente genera la richiesta successiva;
6. seguire il percorso fino alla query SQL.

17. Demo Flask progressive

Prima di leggere l'app palestra completa, puo' essere utile lavorare su esempi piu' piccoli. La cartella:

`labs/flask_interfacce_demo/`

contiene sei demo progressive.

Demo 01: pagina statica

Mostra il caso piu' semplice: Flask restituisce una pagina HTML e il browser costruisce il DOM. Serve a distinguere template sorgente, HTML ricevuto e pagina visualizzata.

Demo 02: template dinamico

Mostra come una lista Python venga trasformata in elementi HTML tramite Jinja. Serve a capire il passaggio:

dati Python -> template -> HTML -> DOM

Demo 03: componenti grafici

Mappa oggetti visibili e tag HTML: pannello, link, campo input, pulsante, form. Serve a costruire il lessico dell'interfaccia.

Demo 04: form GET/POST

Mostra come un form invia dati al server e come Flask li legge con `request.form`. Serve a collegare `name` dei campi HTML e variabili Python.

Demo 05: eventi DOM

Mostra eventi gestiti nel browser: filtro testuale e contatore di click. Serve a distinguere eventi che generano richieste HTTP da eventi locali gestiti con JavaScript.

Demo 06: mini CRUD

Mostra una piccola creazione/eliminazione in memoria. Serve a vedere il pattern:

POST -> modifica stato -> redirect -> GET pagina aggiornata

Queste demo non sostituiscono l'app palestra. La preparano: prima si comprende un concetto isolato, poi lo si riconosce nella struttura reale dell'applicazione.

18. Esercizio mentale: un click

Domanda:

Cosa succede quando l'utente clicca "Istruttore"?

Risposta guidata:

1. nel DOM esiste un link;
2. il link ha un attributo `href`;
3. il browser invia una richiesta GET;
4. Flask abbina URL e route;
5. la route costruisce una pagina;

6. il browser riceve HTML;
7. il DOM viene aggiornato con la nuova pagina.

19. Esercizio mentale: un form

Domanda:

Cosa succede quando l'utente crea un nuovo esercizio?

Risposta guidata:

1. l'utente apre la pagina con il form;
2. il form contiene input con nomi precisi;
3. l'utente compila i valori;
4. il submit produce una richiesta POST;
5. Flask legge `request.form`;
6. Flask costruisce una query INSERT;
7. il database applica vincoli;
8. Flask fa redirect;
9. la lista mostra lo stato aggiornato.

20. Perché questa lezione serve per progettare

Capire Flask non significa solo memorizzare decorator e template.

Significa saper progettare il passaggio tra:

- esigenza dell'utente;
- oggetto visibile;
- evento;
- richiesta HTTP;
- funzione Python;
- operazione sul database;
- risposta HTML.

Questa catena sarà utile anche negli elaborati: prima di scrivere codice, bisogna saper descrivere quali pagine servono, quali dati mostrano, quali eventi generano e quali operazioni applicative invocano.

21. Sintesi

Una interfaccia web Flask elementare si basa su poche idee fondamentali:

- il browser mostra un DOM costruito da HTML;
- gli oggetti grafici corrispondono a tag e attributi;
- gli eventi utente producono richieste;
- Flask associa URL e funzioni Python;
- i template trasformano dati Python in HTML;
- i form portano dati dal browser al server;

- il database resta il punto di persistenza;
- una buona separazione rende il sistema piu' leggibile e modificabile.